The Agentic Crew

How software engineers learn to build with agents

Rasmus Bornhøft Schlünsen

Draft — March 2026

€25

For my beloved children.
You are my crew.

Foreword

I've been a software engineer for a long time. I know what it feels like to have a codebase in your head — to open an editor, navigate to the right file, and just *write* the thing. That muscle memory, built over years of keystrokes and debugging sessions, is real. It's earned.

And now the ground is shifting under our feet.

AI agents can write code, run tests, refactor modules, and ship pull requests. Not perfectly — but well enough that ignoring them is no longer an option. For engineers like us, this raises an uncomfortable question: if the machine can do what I do, what's left for me?

This book is my answer. The craft isn't dying — it's evolving. We're moving from writing every line by hand to something more like directing, orchestrating, and collaborating. Less typing, more thinking. Less editor, more engineering. The skills that got us here — systems thinking, taste, judgement, knowing what to build and why — those matter *more* now, not less.

But the transition is messy. I wrote this book because I'm living it, and I know you are too.

Rasmus Bornhøft Schlünsen
March 2026

Contents

1 Introduction 8
1.1 Why This Book 8
1.2 Who This Is For 8
1.3 How to Read This Book 9
2 Context 10
2.1 The Context Window Is Your Workbench 10
2.2 Local, Sandboxed, Remote: Moving Your Agents Around 11
2.3 Feeding Context Deliberately 12
2.4 Context as Architecture 12
3 What Is an Agent? 14
3.1 The Spectrum 14
3.2 What Makes Something "Agentic" 15
3.3 Agents Are Not Magic 15
3.4 The Human-Agent Relationship 16
3.5 When Agents Fail 16
3.6 The Right Mental Model 17
4 Building My Own IDE: Clovr Code Terminal 18
4.1 What CCT Is 18
4.2 Multiple Sessions, One View 19
4.3 Session Handovers 20
4.4 The Permission Layer 20
4.5 Providers and Models 20
4.6 Git-Aware by Default 21
4.7 Running Anywhere 22
4.8 The iOS Companion 23
4.9 Why I Built It 23
4.10 You Don't Need to Build Your Own 24
5 Guardrails 25
5.1 The Trust Gradient 25
5.2 Permission Scoping 26
5.3 Approval Gates 27

5.4 Building Trust Incrementally . . . 28
5.5 When Guardrails Fail . . . 29
5.6 The Cost of Too Many Guardrails . . . 30
5.7 Environment-Specific Guardrails . . . 31
6 Git as Agent Infrastructure . . . 33
6.1 Small Commits, Always . . . 33
6.2 Let Agents Write Your Commit Messages . . . 34
6.3 Branches as Task Boundaries . . . 34
6.4 Worktrees for Parallel Agents . . . 35
6.5 Reviewing Agent Work Through Diffs . . . 35
6.6 Git History as Documentation . . . 36
6.7 The Git Workflow for Agentic Engineering . . . 37
7 Sandboxes . . . 38
7.1 The Fear Problem . . . 38
7.2 Git Worktrees . . . 38
7.3 Containers . . . 39
7.4 Ephemeral Environments . . . 39
7.5 The Sandbox Mindset . . . 40
8 Testing as the Feedback Loop . . . 41
8.1 The Agent's Eyes . . . 41
8.2 TDD Takes on New Meaning . . . 41
8.3 Speed Matters . . . 42
8.4 Testing Beyond Code . . . 42
8.5 The Virtuous Cycle . . . 43
9 Convention Over Configuration . . . 44
9.1 Why Agents Love Convention . . . 44
9.2 Practical Conventions That Help Agents . . . 44
9.3 CLAUDE.md and Project Documentation . . . 45
9.4 The Compounding Effect . . . 46
10 Local LLMs vs. Commercial LLMs . . . 47
10.1 The Case for Commercial LLMs . . . 47
10.2 The Case for Local LLMs . . . 47
10.3 The Practical Answer: Use Both . . . 48
11 Prompting as Engineering . . . 49
11.1 The Anatomy of a Good Task Prompt . . . 49
11.2 Constraint Specification . . . 50
11.3 Task Decomposition . . . 51

11.4 The Prompt as a Spec 51
11.5 Iteration Over Perfection 52
11.6 Anti-Patterns 53
11.7 Prompting Is a Skill 54
12 Multi-Agent Orchestration 55
12.1 Decomposition Strategies 55
12.2 Branch-per-Agent 56
12.3 The Handover Pattern 57
12.4 The Merge Problem 58
12.5 Orchestration Overhead 59
12.6 A Practical Example 59
13 When Agents Get It Wrong 62
13.1 The Eager Refactorer 62
13.2 The Hallucinated Library 63
13.3 The Infinite Loop 64
13.4 The Confident Wrong Answer 64
13.5 The Context Amnesia 65
13.6 The Dependency Avalanche 66
13.7 The Common Thread 67
14 When Not to Use Agents 68
14.1 The Overhead Tax 68
14.2 Novel Architecture Decisions 69
14.3 Security-Critical Code 69
14.4 When You Need to Learn 70
14.5 Emotionally Charged Decisions 70
14.6 When the Codebase Is Too Messy 71
14.7 The Craft Argument 71
14.8 The Judgment That Matters 72
15 Agentic Teams 73
15.1 The 10x Multiplier Is Real, but Distributed Differently 73
15.2 Code Review Changes 74
15.3 The Junior Engineer Question 74
15.4 Knowledge Distribution 75
15.5 Shared Conventions Matter More 76
15.6 The New Standup 77
15.7 Hiring Changes 77
16 Final Words 79

16.1 What I Believe 79
16.2 What I Got Wrong 79
16.3 The Crew Metaphor, One Last Time 80
16.4 Go Build Something 80

1 Introduction

For twenty years, being a software engineer meant one thing: you open an editor, you write code, you ship it. The tools changed — from Vim to VS Code, from SVN to Git, from bare metal to the cloud — but the fundamental loop stayed the same. You, a keyboard, and a problem.

That loop is breaking.

AI agents don't just autocomplete your code. They read your codebase, reason about it, make changes across files, run your tests, and iterate. They're not replacing the editor — they're replacing the *workflow*. The engineer who used to spend 80% of their time typing is now spending 80% of their time thinking, reviewing, and steering.

This is a book about that transition. Not the hype. Not the existential dread. The practical, daily reality of what it means to go from writing software to *engineering* it — with a crew of agents at your side.

1.1 Why This Book

Because nobody handed us a playbook. The tools showed up fast — Copilot, then Claude, then agents that can run entire tasks autonomously — and we're all figuring it out in real time. Some engineers are thriving. Others are frustrated, skeptical, or quietly terrified.

I wrote this because I needed it myself. A honest look at what's changing, what's staying the same, and how to actually get good at this new way of working.

1.2 Who This Is For

You're a software engineer. You've shipped real things. You know what a production incident feels like at 2am. You're not afraid of the terminal.

But lately, something feels different. Maybe you've tried AI coding tools and found them impressive but chaotic. Maybe you've watched junior engineers ship features faster than you with agent assistance, and it stings a little. Maybe you're excited but don't know where to start.

This book is for you. It assumes you can code. It assumes you've been around. It meets you where you are.

1.3 How to Read This Book

Each chapter is self-contained enough to read on its own, but the book builds a narrative. We start with the shift — what's actually changing and why — then move into the practical: how to work with agents, how to think about code differently, and how to build the judgement that separates a good agentic engineer from someone who just pastes prompts.

Read it front to back, or jump to whatever's bothering you most. Either way works.

2 Context

The single most important skill in agentic engineering isn't prompting. It's context management.

An AI agent is only as good as what it can see. Give it a vague instruction and a blank slate, and it will hallucinate confidently. Give it the right files, the right constraints, the right view of the system — and it will do things that feel like magic. The difference isn't the model. It's you.

Traditional engineering had a version of this too. A senior engineer didn't just write better code — they held more of the system in their head. They knew which files mattered, where the dragons lived, which abstractions were load-bearing and which were decorative. That mental model was the context, and it lived entirely in the engineer's brain.

Now you have to *externalise* it. Your agents can't read your mind. They read files, environment variables, error logs, and whatever you put in front of them. The craft of agentic engineering is learning what to surface, when, and how.

2.1 The Context Window Is Your Workbench

Think of the context window as a physical workbench. It has limited space. You can't dump your entire codebase on it and expect good results. Instead, you lay out the pieces that matter for *this* task: the relevant source files, the failing test, the schema, maybe a snippet of documentation.

A good agentic engineer curates context the way a good surgeon lays out instruments. Nothing unnecessary. Everything within reach.

This means developing instincts for questions like:

- What does the agent need to see to understand this task?
- What will confuse it if I include it?

- Is the context I'm providing *current*, or am I feeding it stale information?

2.2 Local, Sandboxed, Remote: Moving Your Agents Around

Context isn't just about text in a prompt. It's about *where* your agent operates and what it has access to.

2.2.1 The Local Machine

The simplest setup: the agent runs on your machine, reads your files, executes your commands. This is where most people start — tools like Claude Code operating directly in your project directory.

The advantage is immediacy. The agent sees what you see. It can read your code, run your tests, check your git history. The risk is also obvious: it's your machine, your credentials, your production config sitting right there in `~/.env`.

2.2.2 Sandboxed Environments

A more disciplined approach is to give the agent a sandbox — a container, a VM, a worktree. It gets a copy of the code but not your keys. It can break things without breaking *your* things.

This matters more than most people realise. When you let an agent iterate freely — running code, installing packages, modifying files — you want it to do that in a space where a mistake is cheap. A sandboxed agent is a fearless agent, and a fearless agent is a productive one.

Worktrees are an underappreciated tool here. Git worktrees let you spin up an isolated copy of your repo in seconds. The agent works in its own branch, its own directory. If the result is good, you merge it. If not, you delete the worktree and move on. No mess.

2.2.3 Remote Exploration

This is where things get interesting. A skilled agentic engineer doesn't just point agents at local files — they teach agents to *explore* remote systems.

SSH into a staging server to examine logs. Query a database to understand the shape of real data. Curl an API endpoint to see what it actually returns, not what the docs claim it returns. Pull down container logs from a running service.

The agent becomes your scout. You point it at a system and say: "go look around and tell me what you find." But you have to set this up. The agent needs credentials (scoped and temporary), network access, and clear boundaries on what it's allowed to touch.

This is a judgement call — how much access to give, to which systems, with what guardrails. Too little and the agent is useless. Too much and you're one bad prompt away from a production incident. The agentic engineer learns to calibrate this over time.

2.3 Feeding Context Deliberately

The best agentic engineers develop habits around context:

Start with the error. Don't describe the bug — show the agent the stack trace, the failing test output, the log line. Raw context beats paraphrased context every time.

Show, don't tell. Instead of explaining your database schema in prose, give the agent the migration files or the ORM models. Instead of describing the API contract, give it the OpenAPI spec or a curl response.

Prune aggressively. If you're debugging a rendering issue, the agent doesn't need to see your authentication middleware. Every irrelevant file in context is noise that degrades the signal.

Use the filesystem as context. A well-organised project *is* context. Meaningful file names, clear directory structure, a good README — these aren't just for humans anymore. Your agents read them too.

2.4 Context as Architecture

Here's the deeper insight: as you get better at agentic engineering, you start designing your systems *for* context. You write clearer commit mes-

sages because agents read them. You keep functions small because agents work better with focused files. You maintain up-to-date docs because agents treat them as ground truth.

The way you structure your code, your repos, your infrastructure — it all becomes part of the context you're providing to your crew. The engineers who understand this early will build systems that are not just maintainable by humans, but *navigable* by agents.

And that's a competitive advantage that compounds over time.

3 What Is an Agent?

The word "agent" gets thrown around a lot. It's applied to everything from a chatbot that answers questions to a system that autonomously deploys code to production. Before we go further, let's get precise about what we mean — because the distinction matters for how you work with them.

3.1 The Spectrum

Not all AI tools are agents. There's a spectrum, and understanding where a tool falls on it changes how you use it.

Autocomplete is the simplest form. You type, it suggests the next few tokens. GitHub Copilot's inline suggestions, your IDE's tab completion. It's reactive — it responds to what you're typing, one line at a time. Useful, but it doesn't *think*.

A copilot is a step up. It can see more context — your open file, maybe a few related files — and generate larger blocks of code. It can answer questions about your codebase. But it's still fundamentally passive. You ask, it responds. You drive, it assists.

A tool-using agent is where things change. An agent doesn't just generate text — it *acts*. It can read files, write files, run commands, execute tests, make API calls, and inspect the results. Critically, it can do this in a loop: try something, observe the outcome, adjust, try again. It has agency — the ability to take actions in the world and respond to feedback.

An autonomous agent is the far end of the spectrum. Given a high-level goal, it plans its own approach, breaks the work into steps, executes them, handles errors, and delivers a result — potentially without any human interaction along the way.

Most practical agentic engineering today happens in the "tool-using agent" zone. You give the agent a task, it has access to tools, and it works iteratively to complete it. You're in the loop — reviewing, guiding, approving — but the agent is doing the heavy lifting.

3.2 What Makes Something "Agentic"

Three capabilities separate an agent from a fancy chatbot:

Planning. An agent can break a goal into steps. "Add authentication to this app" becomes: read the current codebase, identify the right framework, create the middleware, update the routes, add tests, verify everything passes. A chatbot gives you a code block. An agent gives you a series of actions.

Tool use. An agent can interact with the world. It reads your files, runs your tests, checks your git status, examines error output. Each tool call gives it new information, which it uses to decide what to do next. This feedback loop is what makes agents powerful — they're not generating code in a vacuum, they're generating code and *verifying* it.

Iteration. An agent can try, fail, and try again. It writes a function, runs the tests, sees a failure, reads the error, adjusts the code, runs the tests again. This cycle — act, observe, adjust — is the core of agentic behaviour. A chatbot gives you one shot. An agent gives you a process.

3.3 Agents Are Not Magic

It's important to be clear-eyed about what agents are and what they aren't.

Agents are not sentient. They don't understand your code the way you do. They don't have intuition, taste, or experience. What they have is the ability to process large amounts of text, recognise patterns, and generate plausible next steps — very quickly, very tirelessly, and at a scale that would exhaust any human.

They hallucinate. They make confident mistakes. They sometimes solve the wrong problem beautifully. They can write code that passes all tests but misses the point entirely. They're brilliant interns with infinite energy and no judgment.

This is why the *engineer* matters. The agent provides speed and breadth. You provide direction, judgment, and taste. The combination is more powerful than either alone.

3.4 The Human-Agent Relationship

Think of it like crewing a ship — which is where the title of this book comes from.

You're the captain. You set the course, make the hard calls, and take responsibility for the outcome. The agents are your crew. They handle the work — climbing the rigging, adjusting the sails, keeping watch. They're skilled, they're tireless, and they're good at what they do. But they need direction. They need someone who knows where the ship is going.

A bad captain micromanages every rope. That defeats the purpose of having a crew. A reckless captain gives no direction and wonders why the ship runs aground. The good captain learns which crew members to trust with which tasks, gives clear instructions, checks in at the right moments, and intervenes only when necessary.

That calibration — how much autonomy, for which tasks, with what oversight — is the art of agentic engineering.

3.5 When Agents Fail

They will fail. Understanding *how* they fail helps you build better workflows.

Scope creep. You ask the agent to fix a bug, and it refactors three files, updates the build system, and changes the linting config. Agents are eager to help, and sometimes that eagerness extends beyond what you asked for. This is why small, focused tasks and branch isolation matter.

Hallucinated APIs. The agent generates code that calls a function or library that doesn't exist — or exists in a different version. This is why running tests matters. The agent can't hallucinate its way past a test suite.

Overconfidence. The agent tells you the task is done, and it looks done, but there's a subtle bug that only manifests under specific conditions. This is why you review diffs and don't blindly trust agent output.

Context loss. On long tasks, the agent loses track of earlier decisions. It contradicts itself, rewrites code it already wrote, or forgets constraints you mentioned earlier. This is why small commits and clear context management matter.

Every failure mode has a mitigation, and those mitigations are the chapters that follow: context, guardrails, git, sandboxes, testing, conventions. The principles in this book aren't theoretical — they're direct responses to how agents fail in practice.

3.6 The Right Mental Model

Don't think of agents as tools. Don't think of them as replacements. Think of them as collaborators with a very specific set of strengths and weaknesses.

They're fast where you're slow. They're patient where you're impatient. They can hold more text in working memory than you can. They never get tired, never get frustrated, never have a bad day.

But they don't know what matters. They don't know what the user actually needs. They don't know which technical debt is acceptable and which is a ticking bomb. They don't know when to push back on a requirement. They don't know when the spec is wrong.

That's your job. And it always will be.

4 Building My Own IDE: Clovr Code Terminal

Before we get into the principles, let me tell you about the thing that taught me most of them.

I spent over a hundred hours building Clovr Code Terminal — CCT — my own IDE for agentic engineering. A hundred hours of watching agents write code, break things, fix things, and surprise me. A hundred hours of learning what works, what doesn't, and why. Every chapter that follows in this book grew out of lessons I learned building this tool.

CCT started from a simple frustration: I was running multiple Claude Code sessions in separate terminal windows, constantly switching between them, losing track of which agent was doing what. I wanted a mission control. So I built one — and in the process, I discovered what agentic engineering actually is.

4.1 What CCT Is

CCT is a self-hosted control plane for agentic engineering. It's a single 15MB Go binary that gives you a browser-based dashboard for running, monitoring, and orchestrating multiple AI agent sessions simultaneously. It can run on your local machine or on a server somewhere — a sandbox you SSH into, a VPS, a beefy cloud instance. The agent sessions run where CCT runs, and you control them from any browser.

You start it with one command. No dependencies, no npm install, no Docker required. Just a binary and a browser.

The core idea: you're the engineer, the agents are your crew, and CCT is the bridge where you coordinate them.

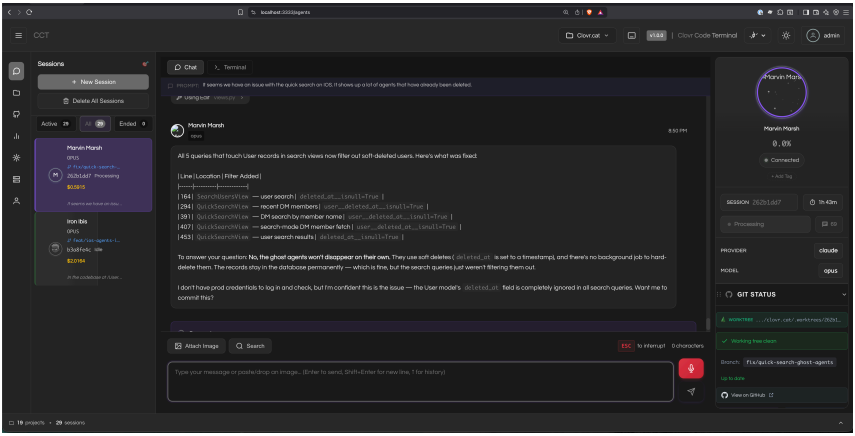

Figure 1: CCT's main dashboard — multiple agent sessions running in parallel, with real-time cost tracking, model selection, and session metadata in the sidebar.

4.2 Multiple Sessions, One View

The sidebar shows all your active sessions. Each one is a separate Claude agent with its own working directory, its own git branch, its own conversation history. You can see at a glance what each agent is doing — which files it's reading, which commands it's running, how many tokens it's consumed, what it's costing you.

This changes how you work. Instead of giving one agent a big task and waiting, you break the work into parallel tracks. One agent refactors the API layer. Another writes tests for the new feature. A third updates the documentation. You glance at the sidebar, check their progress, intervene when needed, and let them run when they're on track.

The sessions persist across restarts. Come back the next morning and your agents' full conversation history is there in SQLite, ready to continue.

4.3 Session Handovers

One of the features I'm most proud of is session handovers. When one agent finishes a task, it can hand its context — a summary of what it did, which files it changed, what's left to do — to a new agent session. The handover is a structured token: time-limited, single-use, carrying just enough context for the next agent to pick up where the last one left off.

This enables pipeline-style workflows. I built a site generator inside CCT that uses three specialist agents in sequence: an orchestrator that plans the work, a designer that creates the visual structure, and an implementer that writes the code. Each one hands off to the next. The result is better than any single agent could produce, because each specialist focuses on what it does best.

The handover system is the "crew" metaphor made concrete. Agents don't work alone — they collaborate through structured context passing.

4.4 The Permission Layer

Every tool call an agent makes — every bash command, every file write, every network request — goes through CCT's permission system. You can set it to approve everything automatically (YOLO mode, useful during rapid prototyping), or require explicit approval for sensitive operations.

There's also a middle ground: always-allow rules. You can whitelist specific commands (`git status`, `npm test`, `ls`) while requiring approval for everything else. Over time, your permission config becomes a reflection of your trust relationship with the agents — how much rope you give them, and where you pull them back.

This is the guardrails chapter in practice.

4.5 Providers and Models

CCT isn't locked to one AI provider. You can configure Claude, DeepSeek, Kimi, GLM, or any Anthropic-compatible endpoint — and switch between them per session, without restarting. Running a quick documentation

task? Point it at a cheaper model. Doing complex multi-file refactoring? Switch to Opus.

This is the local-vs-commercial chapter in practice. The tool doesn't have opinions about which model you use. It gives you the controls to make that choice per task.

4.6 Git-Aware by Default

Every session knows its git context. The sidebar shows the current branch. You can view diffs, compare against main, and track which session modified which files. When you create a new session, CCT can spin up a git worktree automatically — giving the agent its own isolated copy of the repo.

This is the sandboxes chapter in practice. Each agent gets its own workspace, its own branch, its own room to experiment. Good results get merged. Bad results get deleted with the worktree. No mess.

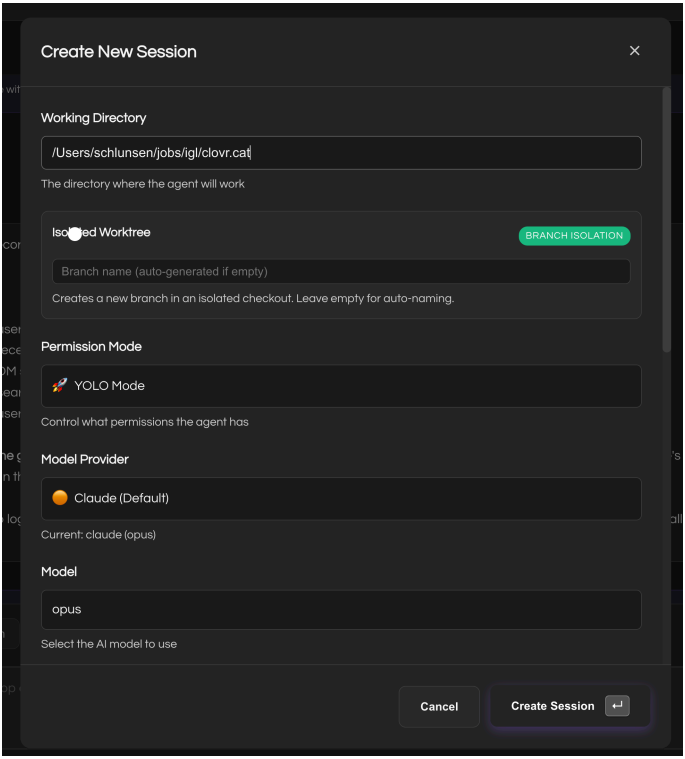

Figure 2: Creating a new session — you pick the working directory, enable worktree isolation, set the permission mode, and choose your model provider. Every principle from this book in one dialog.

4.7 Running Anywhere

CCT is a single binary. This means it runs wherever you need it:

On your laptop — the simplest setup. The agents work in your local projects, with access to your filesystem and tools. Good for daily development.

On a remote server — SSH in, start CCT, point your browser at it. The agents run on server hardware (maybe beefier than your laptop), and

you get a clean sandbox environment. Good for heavy workloads or when you want isolation from your local machine.

In a container — spin up a Docker instance with CCT and your project, give it scoped credentials, let the agents go wild. Destroy it when you're done. Good for CI-style agent work or security-sensitive tasks.

The architecture is intentionally simple: one binary, one SQLite database, one WebSocket connection to your browser. No microservices, no message queues, no distributed systems. It's a tool, not a platform.

4.8 The iOS Companion

Because I'm often away from my desk — picking up the children, making dinner, pretending to be present — I built an iOS app that connects to CCT. It shows me what my agents are doing in real time. If an agent hits a permission request, I can approve it from my phone. If a session finishes, I see the result.

This is a small thing, but it changed my workflow significantly. Agents don't need me watching them constantly. They need me available when they have a question. The mobile app makes that possible without being chained to a screen.

4.9 Why I Built It

I could have used Claude Code in the terminal and been fine. Many excellent engineers do exactly that. But I wanted something that made the *multi-agent workflow* first-class. I wanted to see all my agents at once. I wanted structured handovers. I wanted permission controls I could tune. I wanted it running on a server I could access from anywhere.

More than anything, building CCT taught me what agentic engineering actually *feels* like at the infrastructure level. Every feature I added — sessions, handovers, permissions, worktree isolation — forced me to think deeply about the principles in this book. Context management, guardrails, sandboxing, testing — they're not abstract ideas when you're implementing them in code.

The tool is a reflection of the practice. And the practice made me a better engineer.

4.10 You Don't Need to Build Your Own

Let me be clear: you don't need to build a custom IDE to be a great agentic engineer. Claude Code in a terminal is powerful enough for most workflows. Cursor, Windsurf, and other tools are excellent.

What you *do* need is to understand the principles well enough that you can shape your tools to fit your workflow — not the other way around. Whether that means writing custom `CLAUDE.md` files, configuring permission rules, setting up worktree scripts, or building an entire control terminal from scratch.

The best tool is the one that gets out of your way and lets you focus on the engineering. For me, that's CCT. For you, it might be something else entirely. The principles are the same.

5 Guardrails

Giving an agent power without boundaries isn't bold engineering — it's negligence. Guardrails are what make autonomous agents *usable* in real work. They're the difference between an agent that helps you ship and one that takes down your staging environment at 3am.

And yet guardrails are not a solved problem. They're a living thing — something you tune, test, and argue about. Get them wrong in one direction and your agent is dangerous. Get them wrong in the other direction and your agent is useless. The craft is in finding the line.

5.1 The Trust Gradient

Not every task deserves the same level of autonomy. Reading files? Low risk, let it run. Writing code in a feature branch? Medium risk, check the diff. Running database migrations? High risk, require explicit approval.

The agentic engineer develops an intuition for this gradient. You don't configure it once and forget — you adjust it per project, per task, per phase of development. Early exploration gets a long leash. Production deployment gets a short one.

Think of it like a mixing board. Each category of action has its own slider: file reads, file writes, shell commands, network access, git operations. You push each slider to the level of autonomy you're comfortable with. Some sliders stay low forever. Others you nudge up as confidence builds.

The mistake most people make is treating the gradient as binary — either the agent can do something or it can't. The reality is more nuanced. An agent might be allowed to run `npm test` without asking, but `npm install` requires a prompt. Both are shell commands. The risk profile is completely different.

5.2 Permission Scoping

The principle is the same as in security: least privilege. An agent working on your frontend doesn't need SSH access to your database server. An agent writing unit tests doesn't need your AWS credentials.

In practice this means:

- Scoped API keys with expiration times
- Environment-specific credentials (never share prod keys with a dev agent)
- Read-only access as the default, write access as the exception
- Network isolation where possible — if the agent doesn't need internet, don't give it internet

Tools like Claude Code already have built-in permission systems — allow/deny lists for commands, file access controls, approval prompts for destructive operations. Use them. Don't blindly approve everything because clicking "yes" is faster.

A concrete allowlist might start like this:

```
allowed_commands:
  - git status
  - git diff
  - git log
  - npm test
  - npm run lint
  - cat
  - ls
  - find

denied_commands:
  - rm
  - git push
  - npm publish
  - curl
  - wget
  - docker
```

That's a day-one config. It's conservative. The agent can read, test, and explore — but it can't delete, deploy, or reach the network. You'll feel the friction immediately. The agent will ask for permission to run `npm`

`install` when it needs a dependency. It will ask before creating a file. That's the point.

After a week, you've watched it work. You trust its judgement on file creation. You add `touch` and `mkdir` to the allowlist. After a month, you let it run `npm install` without asking — but only in the project directory, not globally. After three months, you let it push to feature branches but never to `main`.

The allowlist *grows* with your experience. It's a log of trust decisions, and reading an engineer's allowlist tells you exactly how much agentic experience they have.

5.3 Approval Gates

Some actions should always require a human in the loop. Not because the agent can't do them, but because the *consequences* of getting them wrong are too high.

Good candidates for approval gates:

- Any operation that touches production data
- Deleting files or branches
- Installing new dependencies
- Making network requests to external services
- Any git push
- Modifying CI/CD configuration
- Changing environment variables or secrets

The goal isn't to slow the agent down. The goal is to create natural checkpoints where you, the engineer, can verify that the agent's trajectory still matches your intent. A quick glance at a diff takes five seconds. Recovering from a bad deploy takes hours.

Approval gates also serve a second purpose that's easy to miss: they keep you in the loop. An agent that runs for forty minutes without a single checkpoint is an agent you've lost track of. Even if it does everything correctly, you've lost your mental model of what's happening. Regular approval gates force the agent to surface its work, and they force you to stay engaged.

The best gate is one you almost never reject. If you're rejecting approvals constantly, your agent is misconfigured or miscommunicating — and you should fix the root cause, not keep clicking "no."

5.4 Building Trust Incrementally

You wouldn't give a new hire the production database password on their first day. You'd start them with code reviews on every PR. After a few weeks, you'd let them merge to staging without a second pair of eyes. After a few months, they'd have deploy access. After a year, they'd be the one reviewing *your* code.

Agents follow the same trajectory, just compressed.

On day one, your agent runs in a tight sandbox. Every shell command gets approved. Every file write gets reviewed. You're watching it like a hawk — not because you distrust the technology, but because you haven't *calibrated* your expectations yet. You don't know where it's brilliant and where it's brittle.

After a week, patterns emerge. The agent is flawless at writing unit tests. It's solid at refactoring. It occasionally makes questionable choices about dependency management. Now your guardrails reflect that: tests and refactoring run freely, dependency changes get reviewed.

After a month, you've seen a hundred tasks complete successfully. You loosen the leash further. The agent commits to feature branches on its own. It runs the full test suite without asking. It installs dev dependencies when tests require them.

After three months, the agent is like a trusted colleague. You give it a task, check back in an hour, and review the PR. The guardrails are still there — it still can't push to main, still can't touch production, still can't modify CI — but they're invisible for the 95% of work that's routine.

This is the key insight: *guardrails should be barely noticeable for everyday work, and absolutely rigid for exceptional situations.* The agent should flow through its normal tasks without friction, and hit a hard wall the moment it tries something unusual. That wall is where you show up.

The engineers who never loosen guardrails end up abandoning agentic workflows entirely. The agents feel sluggish and frustrating, so they go back to doing everything manually. The engineers who loosen guardrails *too fast* get burned by an agent that does something unexpected. The sweet spot is a steady, evidence-based ratchet.

5.5 When Guardrails Fail

They will. An agent will misunderstand a constraint, find a creative workaround to a limitation, or encounter an edge case your guardrails didn't anticipate. This is normal.

Here's a real scenario. An engineer had `rm` and `rm -rf` on the deny list — sensible enough. The agent needed to undo some changes to a set of files. It couldn't delete them. So it ran `git checkout -- .` which *was* on the allowlist, because checking out files from git sounds harmless. The result? Every uncommitted change in the working directory — including the engineer's own in-progress work on other files — was wiped clean. The agent solved its narrow problem and created a much larger one.

The lesson isn't that `git checkout` should be denied. It's that guardrails are *defense in depth*, not a single wall. You need multiple layers:

- **The allowlist** catches the obvious dangerous commands.
- **The sandbox** (a worktree, a container) limits the blast radius.
- **The commit history** lets you recover when something slips through.
- **Your own review** catches the things that no automated rule would flag.

No single layer is sufficient. An agent that's blocked from running `rm` will find another way to delete data if that's what it thinks the task requires. It's not being malicious — it's being *resourceful*. The same creativity that makes agents useful is the thing that makes single-layer guardrails insufficient.

The response isn't to remove autonomy — it's to improve the guardrails and add layers. Every failure is a signal. Treat it like a bug: understand what happened, add a check, and move on. Over time, your guardrail

configuration becomes a reflection of hard-won experience, not unlike how a `.gitignore` grows with a project.

The best agentic engineers don't fear agent mistakes. They build systems where mistakes are caught early, contained quickly, and learned from automatically.

5.6 The Cost of Too Many Guardrails

There's a failure mode that nobody talks about because it doesn't look like a failure. It looks like caution.

You configure your agent with approval gates on everything. Every file write: approval. Every shell command: approval. Every git operation: approval. The agent asks, you approve, the agent asks, you approve. Fifteen minutes into a task, you've clicked "yes" forty times and you're exhausted.

At this point, you're not using an agentic workflow. You're using a very slow, very annoying autocomplete. The agent does the typing and you do the thinking — which is exactly the same division of labour you had *before* agents, except now there's a confirmation dialog in the way.

Over-constrained agents produce two outcomes, both bad. Either the engineer gives up and stops using agents, concluding they're "not ready yet." Or — worse — the engineer starts approving everything without reading it, because the approval fatigue has trained them to click "yes" reflexively. Now you have guardrails that *feel* safe but provide zero actual protection.

The skill is in finding the sweet spot. You want guardrails tight enough to catch genuine mistakes, and loose enough that the agent can *flow*. A good heuristic: if you're approving the same type of action more than five times in a session, it should probably be on the allowlist.

Another heuristic: track how often your approvals actually reject something. If you've approved five hundred actions and rejected three, your guardrails are too aggressive for those action types. If you've approved

fifty and rejected ten, they're well-calibrated — those ten rejections are the ones that matter.

The goal is an agent that works like a skilled contractor. It shows up, does the job, and checks in with you at meaningful milestones. Not after every hammer swing.

5.7 Environment-Specific Guardrails

Your development laptop, your staging server, and your production environment are three completely different risk profiles. Your guardrails should reflect that.

Local development is where you give agents the most freedom. The agent can install packages, run arbitrary commands, modify any file, and execute tests — because the worst case is that you `git reset` and start over. Your local machine is a playground. Let the agent play.

Even here, there are limits. The agent shouldn't have access to production credentials. It shouldn't be able to push to `main`. It shouldn't be able to publish packages. But within the bounds of "local development on a feature branch," let it move fast.

Staging tightens the screws. The agent can deploy to staging — but with approval. It can read staging logs and query staging databases — but not modify data. It can run integration tests against staging services — but not reconfigure those services. Staging is where you verify that the agent's work survives contact with a real environment, and the guardrails reflect the higher stakes.

Production is a different animal entirely. The most honest advice: your production agent should be read-only, if it exists at all. Let it query logs. Let it read metrics. Let it investigate incidents by pulling data. But the moment it needs to *change* something in production — a config value, a database record, a running service — that's a human decision, full stop.

Some teams allow agents to execute pre-approved runbooks in production: restart a service, scale up replicas, roll back to a known-good deploy. These are tightly scoped, well-tested operations with clear rollback paths.

That's reasonable. But "let the agent figure out how to fix the production incident" is not a guardrail configuration — it's a prayer.

The pattern is simple: the closer you are to real users and real data, the tighter the guardrails get. Your local agent is a collaborator. Your staging agent is a supervised worker. Your production agent is a read-only observer.

Set this up once, and it becomes invisible. The agent adjusts its behaviour based on the environment it's operating in. It moves fast locally, checks in on staging, and goes hands-off in production. That's not friction — that's engineering.

6 Git as Agent Infrastructure

You already know Git. You've been committing, branching, and merging for years. But in agentic engineering, Git isn't just version control — it's the backbone of your entire workflow. It's your undo button, your parallel execution framework, your review interface, and your documentation system all at once.

Most engineers use maybe 20% of what Git offers. Agentic engineering demands the other 80%.

6.1 Small Commits, Always

The single most important Git habit for agentic engineering: commit small, commit often.

When an agent makes changes, you want to be able to review each logical step independently. A commit that says "refactored authentication, updated tests, fixed the navbar, and changed the database schema" is impossible to review and impossible to roll back partially. Four separate commits — each doing one thing — give you surgical control.

This matters more with agents than with human developers, because agents move fast. An agent can make twenty file changes in thirty seconds. If those changes are bundled into one commit, and something breaks, you're untangling a mess. If they're in five clean commits, you revert the one that broke things and keep the rest.

Train yourself — and your agents — to commit at natural boundaries:

- After each logical change, not after each session
- Before switching to a different concern
- After tests pass, capturing a known-good state
- Before attempting something risky, creating a save point

6.2 Let Agents Write Your Commit Messages

This is one of the easiest wins in agentic engineering, and it's almost embarrassingly simple: let the agent write the commit message.

Think about it. The agent just made the changes. It knows exactly what it modified, why it modified it, and what the intent was. It has the full diff in context. It will write a more accurate, more descriptive commit message than you would — because you'd be summarising from memory, and the agent is summarising from facts.

A typical human commit message at 11pm: "fix auth bug"

A typical agent commit message: "fix session expiry race condition when WebSocket disconnects during OAuth token refresh — the cleanup goroutine was running before the token exchange completed, leaving orphaned sessions in the database"

The second one is useful six months from now when someone — human or agent — is trying to understand why that code exists. The first one is noise.

Make this a habit. After the agent completes a task, ask it to commit with a descriptive message. Or configure your workflow so it happens automatically. The quality of your git history will improve overnight.

6.3 Branches as Task Boundaries

Every agent task gets its own branch. This is non-negotiable.

The branch serves multiple purposes:

- **Isolation.** The agent's changes don't affect your main branch until you explicitly merge them.
- **Review scope.** When you're done, you review a single PR — the diff between the branch and main. This is a workflow every engineer already knows.
- **Rollback.** If the whole thing is wrong, you delete the branch. Clean. No surgery required.
- **Parallel work.** Multiple agents on multiple branches, working simultaneously, never stepping on each other.

Name your branches descriptively: `agent/refactor-auth-middleware`, `agent/add-user-tests`, `agent/fix-sidebar-rendering`. When you look at your branch list, you should see a manifest of everything your agents are working on.

6.4 Worktrees for Parallel Agents

Branches alone aren't enough for true parallel work. If two agents are on different branches but sharing the same working directory, they'll fight over the filesystem. Git worktrees solve this.

A worktree is a separate checkout of your repo — a different directory, on a different branch, sharing the same `.git` history. Creating one takes seconds:

```
git worktree add ../my-project-feature feature-branch
```

Now you have two directories: your main checkout and the worktree. Each agent gets its own worktree, its own branch, its own filesystem. They can both run tests, modify files, and build — simultaneously, without conflicts.

When the work is done:

- Good result → merge the branch, remove the worktree
- Bad result → remove the worktree, delete the branch
- Need to iterate → keep the worktree, continue the conversation

This is the cheapest sandbox you can build. No containers, no VMs, no cloud resources. Just Git.

6.5 Reviewing Agent Work Through Diffs

Your primary interface for reviewing agent work isn't reading code — it's reading diffs.

This is a subtle but important shift. When you write code yourself, you review it as you write. When an agent writes code, you review it after the fact. And the most efficient way to do that is through the diff against your base branch.

Develop your diff-reading skills:

- **Start with the test changes.** If the agent wrote or modified tests, read those first. They tell you what the agent thinks the code should do. If the tests match your intent, the implementation is probably fine.
- **Look for scope creep.** Did the agent change files you didn't expect? Unrelated formatting changes? Extra dependencies? These are red flags.
- **Check the boundaries.** Function signatures, API contracts, database schemas — changes to interfaces have outsized impact. Review these carefully.
- **Trust but verify.** If the diff is large, don't read every line. Spot-check the critical paths, make sure the tests are meaningful, and run the suite yourself.

The goal isn't to read every line the agent wrote — that defeats the purpose. The goal is to verify that the agent's changes match your intent and don't introduce problems. Diffs make this fast.

6.6 Git History as Documentation

Here's the insight most engineers miss: agents read your git history. When an agent is trying to understand why code exists, how a feature evolved, or what approach was tried before, it looks at `git log` and `git blame`.

This means your commit history is documentation. Not the kind you write in a wiki — the kind that's embedded in the code itself, permanently, with perfect provenance.

Good commit messages compound over time. Six months from now, when an agent is working on your codebase, it will read those messages to understand context. The difference between a history of "fix bug" and "fix race condition in session cleanup" is the difference between an agent that understands your codebase and one that's guessing.

This also applies to PR descriptions, branch names, and merge commit messages. Every piece of text you attach to your Git history is context for future agents. Write accordingly.

6.7 The Git Workflow for Agentic Engineering

Putting it all together, here's the workflow:

1. Create a branch for the task
2. Optionally create a worktree for isolation
3. Point the agent at the worktree
4. Let it work — committing at natural boundaries
5. Agent writes descriptive commit messages
6. Review the diff against main
7. Merge if good, discard if not

This workflow is fast, safe, and scales to multiple parallel agents. It uses Git features that have existed for years — branches, worktrees, diffs — but combines them in a way that's purpose-built for agentic engineering.

The best part: you already know all of this. You've been using Git for years. Agentic engineering doesn't require new tools — it requires using your existing tools more deliberately.

7 Sandboxes

A sandbox is a gift you give your agent: the freedom to be wrong.

When an agent operates in a sandbox, it can try things without consequences. Install a weird dependency. Rewrite a module from scratch. Run a script that might crash. If it works, great — you pull the result out. If it doesn't, you throw the sandbox away. No cleanup, no rollback, no damage.

This isn't just a safety measure. It fundamentally changes how productive an agent can be.

7.1 The Fear Problem

Without a sandbox, every agent action carries risk. The agent hesitates (or rather, you hesitate to let it act). You add more constraints, more approval gates, more guardrails — and soon the agent is barely more useful than a fancy autocomplete.

Sandboxes solve this by making the *cost of failure* essentially zero. And when failure is cheap, experimentation is free. An agent in a sandbox can try three different approaches to a problem, run them all, and let you pick the winner. That's a workflow that's impossible when every action is irreversible.

7.2 Git Worktrees

For code-focused work, git worktrees are the lightest sandbox you can build. A worktree gives you a full copy of your repo in a separate directory, on its own branch, in seconds.

The workflow looks like this:

1. Create a worktree for the task

2. Point the agent at it
3. Let it work — commits, file changes, test runs, whatever it needs
4. Review the result
5. Merge if good, delete the worktree if not

No containers to build, no VMs to boot. Just git. This is especially powerful when you're running multiple agents in parallel — each gets its own worktree, its own branch, its own isolated workspace.

7.3 Containers

For tasks that go beyond code — installing system packages, running services, testing infrastructure changes — containers are the natural sandbox. Docker gives you a reproducible, isolated environment that you can spin up in seconds and destroy just as fast.

The key is to make your project container-friendly. A good `Dockerfile` and `docker-compose.yml` aren't just for deployment anymore — they're agent infrastructure. When your project can boot in a container with one command, you've given every agent the ability to work in a clean room.

7.4 Ephemeral Environments

The most sophisticated sandboxes are cloud-based ephemeral environments — short-lived preview deployments that spin up per branch or per task. Services like Railway, Fly.io, or cloud dev environments give you a full running stack that's completely disposable.

This matters for integration testing. An agent can deploy its changes to an ephemeral environment, run end-to-end tests against real infrastructure, verify everything works, and then you decide whether to promote it to staging. The agent never touches your real environments.

7.5 The Sandbox Mindset

The deeper lesson isn't about tools — it's about designing your workflow around disposability. If your development environment takes an hour to set up, sandboxes are impractical. If it takes thirty seconds, they're natural.

Invest in fast, reproducible setup. Invest in infrastructure as code. Invest in making your project easy to boot from scratch. These investments pay for themselves every time an agent needs a safe space to work — which, as you get better at agentic engineering, is constantly.

8 Testing as the Feedback Loop

In traditional development, tests verify that your code works. In agentic engineering, tests do something more fundamental: they tell the agent whether it succeeded.

This changes everything about how you think about testing.

8.1 The Agent's Eyes

An agent can't look at a UI and tell if it looks right. It can't feel whether an API response is "fast enough." It can't intuit whether a refactor preserved the subtle behaviour that users depend on. What it *can* do is run your test suite and read the results.

Tests become the agent's primary feedback mechanism. Green means "keep going." Red means "try again." No tests means the agent is flying blind — guessing whether its changes work, with no way to verify.

This is why untested codebases are hard to work with agentically. It's not just a quality problem — it's an information problem. Without tests, the agent has no signal.

8.2 TDD Takes on New Meaning

Test-Driven Development was always a good idea. With agents, it becomes a superpower.

Write the test first. Describe the expected behaviour. Then hand it to the agent and say: "make this pass." The agent now has a clear, unambiguous success criterion. It can iterate — write code, run the test, read the failure, adjust, repeat — in a tight loop that takes seconds per cycle.

This is fundamentally different from asking an agent to "build a login system." That's vague. But "make these fourteen tests pass" is precise.

The agent knows exactly what success looks like, and it can measure its own progress.

The agentic engineer learns to express intent through tests. Every test is a contract. Every assertion is a requirement. The better your tests, the better your agents perform.

8.3 Speed Matters

When an agent iterates in a test loop, the speed of your test suite directly affects productivity. A test suite that takes ten minutes to run means the agent waits ten minutes between attempts. A test suite that takes ten seconds means the agent can try dozens of approaches in the time it takes you to get coffee.

This creates a strong incentive to:

- Keep unit tests fast and isolated
- Separate fast tests from slow integration tests
- Use watch modes that re-run only affected tests
- Invest in test infrastructure the same way you invest in CI

Fast tests aren't just a developer experience improvement anymore. They're agent infrastructure.

8.4 Testing Beyond Code

Agents don't just write application code. They write infrastructure configs, deployment scripts, documentation, and more. Each of these can — and should — have some form of automated verification.

Linting for config files. Link checkers for documentation. Schema validation for API contracts. Type checking as a fast feedback loop. Every automated check you add is another signal the agent can use to self-correct.

The agentic engineer thinks about testability broadly: not "does my function return the right value?" but "can I automatically verify that this change is correct?"

8.5 The Virtuous Cycle

Here's what happens when you combine good tests with sandboxed agents: the agent makes a change, runs the tests, sees a failure, adjusts, runs again — all in its own isolated environment, all without your intervention. By the time it comes back to you, it's already verified its own work.

You review a diff that you know passes all tests. Your job shifts from "check if this works" to "check if this is the right approach." That's a much better use of your time.

This is the virtuous cycle of agentic engineering: better tests lead to more autonomous agents, which lead to faster iteration, which gives you more time to write better tests.

9 Convention Over Configuration

There's an old principle in software: convention over configuration. Make the default the right thing. Reduce the number of decisions that need to be made. When everyone follows the same patterns, the code explains itself.

This principle was always useful for human teams. For agentic engineering, it's essential.

9.1 Why Agents Love Convention

An agent navigating an unfamiliar codebase does the same thing a new hire does: it looks for patterns. Where do tests live? How are files named? What's the import convention? Where's the config?

If your project follows strong conventions, the agent picks up the patterns quickly and produces code that fits in. If every file is a snowflake — different naming, different structure, different styles — the agent flounders. It doesn't know which pattern to follow, so it invents its own, and the result feels foreign.

Convention is *implicit context*. It's information the agent absorbs from the structure of your code without you having to explain it.

9.2 Practical Conventions That Help Agents

Consistent file naming. If your API routes live in `routes/`, your models in `models/`, and your tests in `__tests__/` — the agent can navigate your project without a map. Name files after what they contain. Keep it boring.

Formatters and linters. Tools like Prettier, ESLint, `gofmt`, or Ruff aren't just for code style — they're guardrails that ensure agent-generated code matches your project's standards automatically. Run them on save, run them in CI, make them non-negotiable.

Standard project layout. Whether it's the Go standard layout, Rails conventions, or your team's own structure — pick one and stick to it. A `justfile` or `Makefile` at the root that lists the standard commands (`build`, `test`, `lint`, `dev`) gives agents an entry point into any project.

Small, focused files. Agents work better with files under a few hundred lines. When a file contains one concern — one component, one module, one set of related functions — the agent can read it, understand it, and modify it without wading through unrelated code.

Descriptive commit messages. Agents read git history. A commit that says "fix bug" teaches nothing. A commit that says "fix race condition in session cleanup when WebSocket disconnects during auth" gives the agent context about what was important, what was fragile, and how the team thinks about problems.

9.3 CLAUDE.md and Project Documentation

A growing convention in agentic engineering is the `CLAUDE.md` file (or equivalent): a document at the root of your project that tells the agent what it needs to know. How to build. How to test. What the architecture looks like. What to avoid.

This isn't a README — it's a briefing document. Short, current, and focused on what an agent needs to be productive in this specific codebase. Think of it as the conversation you'd have with a contractor on their first day: "here's the project, here's how we do things, here's where the bodies are buried."

Keep it updated. An outdated `CLAUDE.md` is worse than none at all, because the agent will trust it.

9.4 The Compounding Effect

Every convention you establish, every standard you enforce, every piece of documentation you maintain — it all compounds. Each one makes agents slightly more effective, slightly more autonomous, slightly more likely to produce code that fits your project like a glove.

The engineers who invest in convention early don't just have cleaner codebases. They have codebases that are ready for the agentic future. Their agents produce better results. Their reviews are faster. Their iteration cycles are tighter.

Convention isn't glamorous work. But it's the foundation that everything else in this book builds on.

10 Local LLMs vs. Commercial LLMs

Where does the model run? It sounds like an infrastructure question, but it's really a question about trust, cost, and capability — and every agentic engineer has to answer it for themselves.

10.1 The Case for Commercial LLMs

Commercial models — Claude, GPT, Gemini — are where the frontier is. They have the largest context windows, the best reasoning, and the most consistent tool use. For serious agentic engineering work, they're hard to beat.

The trade-off is straightforward: your code leaves your machine. Every file you feed into context, every prompt you write, every error log you share — it travels over the wire to someone else's infrastructure. For personal projects and many companies, this is fine. For defence contractors, healthcare, or anyone bound by strict data residency rules, it's a non-starter.

Cost is the other factor. When you're running agents that iterate in loops — reading, writing, testing, retrying — token usage adds up. A single complex task can burn through thousands of calls. At scale, this becomes a real line item.

10.2 The Case for Local LLMs

Running a model locally — Llama, Mistral, Qwen, or any of the open-weight models via Ollama, llama.cpp, or vLLM — gives you something commercial APIs can't: complete privacy and zero marginal cost per token.

Your code never leaves your machine. You can feed it proprietary source, internal documents, production logs — anything — without thinking twice about data policies. And once the model is downloaded, every inference is free. Run it a thousand times, run it all night, nobody sends you a bill.

The trade-off is capability. Local models are smaller, less capable, and have shorter context windows. A 70B parameter model running on a beefy MacBook is impressive, but it's not going to match a frontier commercial model on complex multi-file reasoning tasks. You'll find it handles focused, well-scoped tasks well — writing a single function, explaining a piece of code, generating tests for a known interface — but struggles when it needs to hold a large system in its head.

10.3 The Practical Answer: Use Both

Most experienced agentic engineers land on a hybrid approach. Local models for the high-volume, low-stakes work: quick code generation, boilerplate, commit messages, documentation drafts, and anything touching sensitive data. Commercial models for the hard stuff: complex debugging, multi-file refactoring, architectural reasoning, and tasks where you need the best judgement available.

The key insight is that this isn't a religious choice — it's an engineering one. You match the model to the task the same way you'd choose between SQLite and Postgres. The agentic engineer who can fluently move between local and commercial models, picking the right tool for the job, will consistently outperform someone locked into either camp.

And the landscape is shifting fast. The gap between local and commercial models shrinks with every release. The setup that's right today might look different in six months. Stay flexible.

11 Prompting as Engineering

There's no secret syntax. No magic incantation that makes an agent produce perfect code. Prompting is *communication* — and you already know how to communicate.

If you've ever written a good bug report, you know how to prompt. If you've ever written a design doc that a teammate could implement without asking you twenty questions, you know how to prompt. If you've ever filed a Jira ticket that didn't come back as something completely different from what you wanted — you know how to prompt.

The skills transfer directly. Clarity, specificity, context, constraints. The same things that make human collaboration efficient make agent collaboration efficient. The difference is that agents won't ask clarifying questions when your prompt is vague. They'll just guess. And they'll guess confidently.

11.1 The Anatomy of a Good Task Prompt

A good prompt has three parts: *what* you want done, *why* it matters, and *how* success looks. Most people only provide the first, and even that is usually vague.

Consider the difference:

Bad: "Fix the auth bug."

This tells the agent almost nothing. Which auth bug? Where does it manifest? What's the expected behaviour? The agent will go hunting through your codebase, form a theory about what you might mean, and apply a fix that might be entirely wrong. You've turned a five-minute fix into a twenty-minute review of something you didn't ask for.

Good: "The login endpoint returns 401 for valid tokens when the session cache is cold. The bug is likely in `middleware/auth.go` in the

`validateSession` function. The test in `auth_test.go:TestColdCacheLogin` reproduces it. Fix the bug and make sure all existing auth tests still pass."

This is a different animal entirely. The agent knows the symptom, the suspected location, and how to verify the fix. It can go straight to the relevant code, understand the problem, and validate its solution — all without guessing.

The pattern is simple. *What* is broken or needed. *Where* to look. *How* to verify. Every minute you spend making your prompt precise saves you five minutes reviewing the wrong output.

11.2 Constraint Specification

Telling an agent what to do is only half the job. Telling it what *not* to do is equally important.

Agents are eager. They optimise for solving the problem you described, and they'll happily refactor your entire module, add three new dependencies, and change the public API to do it. That's not malice — it's an optimiser doing what optimisers do. Your job is to set the boundaries.

Useful constraints look like this:

- "Don't modify the public API surface."
- "Keep the existing test structure — add new test cases, don't reorganise."
- "Don't add new dependencies."
- "Stay within the existing error handling patterns in this codebase."
- "Don't change any files outside of the `services/` directory."

Think of constraints as the guardrails on a bridge. The agent can drive anywhere within the lanes, but it can't go over the edge. Without guardrails, you get creative solutions that technically work but create maintenance nightmares. With them, you get solutions that fit your codebase like they were always there.

The more experienced you become with agentic engineering, the more your prompts are defined by their constraints rather than their instructions. You learn which freedoms lead to good outcomes and which lead to chaos.

11.3 Task Decomposition

Here's a common mistake: asking an agent to build something large in a single prompt. "Build a user dashboard with real-time metrics, role-based access, and export to CSV." That's not a prompt — that's a project. And projects need to be broken into tasks.

Task decomposition is the practice of splitting big requests into small, *verifiable* steps. Each step has a clear input, a clear output, and a clear way to check whether it worked.

Instead of "build a user dashboard," you write:

1. Create the data model for dashboard metrics in `models/dashboard.go` with the schema defined in the design doc. Write unit tests for the model validation.
2. Build the API endpoint `GET /api/dashboard` that returns metrics for the authenticated user. Write integration tests.
3. Add role-based filtering so admin users see all metrics and regular users see only their own. Update the existing tests to cover both roles.
4. Build the React component that displays the dashboard data. Use the existing `DataTable` component for the metrics grid.

Each step is a self-contained prompt. Each has a verifiable outcome. Each builds on the verified output of the previous step. If step two goes sideways, you catch it before you've wasted time on step three.

This isn't just good prompting — it's good engineering. You're applying the same decomposition skills you'd use when planning a sprint or breaking down a pull request. The unit of work is small enough to review, small enough to test, and small enough to throw away if it's wrong.

11.4 The Prompt as a Spec

The best prompts I've seen read like miniature design documents. They describe the desired outcome, not the implementation steps. They list the constraints. They define acceptance criteria. They provide just enough context for the agent to make good decisions without drowning it in irrelevant information.

Here's what a prompt-as-spec looks like:

"Add rate limiting to the `/api/search` endpoint. Use the existing `RateLimiter` middleware in `middleware/ratelimit.go`. Set the limit to 100 requests per minute per authenticated user, and 20 per minute for unauthenticated requests. Return a 429 status with a `Retry-After` header when the limit is exceeded. Add tests for both the authenticated and unauthenticated paths, including the edge case where a user hits exactly the limit. Don't modify the rate limiter middleware itself — just configure and apply it."

That's a spec. An agent can implement this without ambiguity. A human reviewer can check the result against the requirements. The desired outcome is clear, the constraints are explicit, and the verification criteria are defined.

Writing prompts this way takes practice. It also takes discipline — the discipline to think through what you actually want before you start typing. But that discipline pays dividends. A well-specified prompt produces a result you can merge. A vague prompt produces a result you have to rewrite.

11.5 Iteration Over Perfection

Your first prompt won't be perfect. That's fine. Prompting is an iterative process, and the skill isn't in writing the perfect prompt — it's in *reading the output*, understanding where the communication broke down, and refining.

When an agent produces something wrong, resist the urge to blame the tool. Instead, ask yourself: what did I fail to communicate? Did I leave out a constraint? Was the context insufficient? Did I assume knowledge the agent didn't have?

This is debugging — but instead of debugging code, you're debugging your own communication. The error message is the agent's output. The stack trace is your prompt. Somewhere in there is the miscommunication, and finding it makes your next prompt better.

Experienced agentic engineers develop a feedback instinct. They see the agent's output and immediately know which part of their prompt caused the deviation. "Ah, I said 'handle errors' but didn't specify *which* errors or *how* to handle them. Of course it threw a generic catch-all in there."

Each iteration tightens the loop. First prompt gets you 70% of the way. A follow-up correction gets you to 90%. A final refinement gets you to done. Over time, your first prompts get better, and you need fewer iterations. But you never need zero.

11.6 Anti-Patterns

Some prompting habits consistently produce poor results. Learn to recognise them.

Being too vague. "Make this code better." Better how? Faster? More readable? More maintainable? The agent will pick *something* to improve, and it probably won't be the thing you had in mind. If you can't articulate what "better" means, you're not ready to prompt.

Being too prescriptive. The opposite failure. "On line 47, change the variable name from `x` to `count`, then add an if statement on line 48 that checks if count is greater than zero, then…" You're writing the code in English and asking the agent to translate. That's slower than writing the code yourself. Describe the *outcome*, not the keystrokes.

Context dumping. Pasting your entire codebase, all your design docs, and a transcript of your last three team meetings into the prompt. More context is not always better. Irrelevant context is noise, and noise drowns signal. Give the agent what it needs — file paths, function names, the specific behaviour you want — and trust it to explore from there.

Kitchen-sink prompts. "Fix the auth bug, also refactor the database layer, and while you're at it update the README and add TypeScript types to the API client." These are four separate tasks jammed into one prompt. The agent will attempt all of them, do none of them well, and produce a diff so large that reviewing it takes longer than doing the work yourself. One prompt, one task.

Assuming shared context. "Do it the same way we did the payments module." The agent doesn't remember your last session. It doesn't know what "we" decided in standup. Every prompt starts from scratch. Provide the context explicitly, every time.

11.7 Prompting Is a Skill

Prompting is not a parlour trick. It's not about discovering the one weird phrase that unlocks better output. It's a communication skill — and like all communication skills, it improves with practice, feedback, and deliberate attention.

The engineers who get the most out of agentic tools are the ones who treat prompting with the same rigour they apply to writing code. They think before they type. They specify before they implement. They verify before they move on.

That's not a new skill. That's just engineering.

12 Multi-Agent Orchestration

One agent is powerful. Multiple agents working together is something else entirely.

Think of it this way. A single carpenter can build a shed. But a house? You need an electrician, a plumber, a roofer — specialists working in parallel, each focused on what they do best, coordinating just enough to stay out of each other's way. The house goes up faster and the work is better than one person trying to do everything.

Agentic engineering works the same way. A single agent on a complex task will grind through it sequentially — plan, implement, test, debug, iterate. It works, but it's slow. Three agents, each handling a well-scoped piece of the problem, can finish in a third of the time. Sometimes less, because focused agents make fewer mistakes than one agent juggling too many concerns.

But there's a catch. Parallel agents require *coordination*. And coordination has a cost. This chapter is about when to pay that cost, and how to pay it well.

12.1 Decomposition Strategies

The hardest part of multi-agent work isn't running multiple agents. It's deciding how to split the work.

The golden rule: tasks must have *minimal coupling*. If agent A's work depends on agent B's output, they can't run in parallel — they're sequential, and you should treat them that way. The art is finding seams in your work where you can cut cleanly.

There are three reliable decomposition patterns.

By layer. One agent handles the backend API, another builds the frontend component, a third writes the tests. This works well because

layers have natural boundaries — a defined interface between them. As long as you agree on the API contract up front, each agent can work independently.

By feature. If you're building three independent features, give each to a separate agent. This is the simplest decomposition. The features touch different files, different directories, different concerns. Merge conflicts are rare.

By concern. One agent refactors, another writes tests for the refactored code, a third updates the documentation. This is a sequential pattern — more pipeline than parallel — but it lets each agent focus on a single type of thinking. The refactoring agent doesn't have to context-switch into test-writing mode. It just refactors, and hands off.

The decomposition you choose depends on the shape of the work. But the principle is constant: *find the seams, cut along them, minimise the surface area where agents need to agree.*

12.2 Branch-per-Agent

Every agent gets its own branch. We covered this in the Git chapter. In multi-agent work, it becomes absolutely essential.

But branches alone aren't enough. Two agents on different branches sharing the same working directory will fight over the filesystem — they'll overwrite each other's files, corrupt each other's builds, break each other's test runs. You need *worktrees*.

Each agent gets its own git worktree: a separate directory, on its own branch, with its own copy of the codebase. The agents share history but nothing else. They can build, run tests, install dependencies, and make a mess — all without affecting each other.

The setup is quick:

```
git worktree add ../project-api agent/api-endpoint
git worktree add ../project-frontend agent/frontend-component
git worktree add ../project-tests agent/integration-tests
```

Three directories. Three branches. Three agents. Full isolation. This is the sandbox model from earlier chapters made concrete for multi-agent work. When an agent finishes, you review its branch, merge if it's good, and remove the worktree. If the work is bad, you throw the whole thing away. Zero cost.

12.3 The Handover Pattern

Not all multi-agent work is parallel. Sometimes agents work *sequentially*, each building on the last. Agent A plans. Agent B implements. Agent C reviews.

This is the handover pattern, and it's powerful — but only if the handover itself is clean.

The problem with sequential agents is context loss. Agent A has a rich understanding of the problem by the time it finishes planning. Agent B starts cold. If you just tell Agent B "implement the plan," it's going to miss nuance. It'll make different assumptions. It'll solve a slightly different problem than the one Agent A planned for.

The fix is *structured handover*. Agent A doesn't just plan — it produces an artefact that captures its reasoning. This can take several forms:

- **A summary document.** A markdown file dropped into the repo: `PLAN.md`. It describes what needs to happen, what trade-offs were considered, what was rejected and why. Agent B reads this before writing a line of code.
- **A set of files.** Agent A creates stub files — empty functions with docstrings, interface definitions, type signatures. Agent B fills them in. The stubs *are* the handover.
- **A structured prompt.** Agent A's output becomes Agent B's input, formatted as a detailed task description with acceptance criteria. You paste it directly into Agent B's context.

The key is that the handover must be *explicit and complete*. No implicit assumptions. No "the next agent will figure it out." Every decision, every constraint, every edge case — written down.

This takes discipline. But it's the same discipline you'd apply when writing a ticket for a human colleague on a different continent and a different timezone. If you wouldn't trust a verbal handover, don't trust an implicit one between agents.

12.4 The Merge Problem

Here's where multi-agent work gets genuinely hard.

Three agents worked in parallel. Each produced a clean, tested branch. Now you need to merge them all into main. Sometimes this is painless — three branches touching completely different files merge without a single conflict. Beautiful.

Sometimes it's not. Agent A changed the database schema. Agent B added a migration that assumes the old schema. Agent C updated the API handler that both A and B also touched. Now you have a three-way conflict and no single agent has the full picture.

There are three strategies for dealing with this.

Prevention. The best merge conflicts are the ones that never happen. When you decompose work, think about file ownership. Assign different directories, different modules, different files to different agents. If agents never touch the same file, they can never conflict. This is the cheapest strategy and you should use it whenever possible.

The merge agent. When conflicts do arise, spin up a new agent whose sole job is to merge. Give it the branches, the conflicts, and the context from each agent's work. A good merge agent can resolve most conflicts automatically — it reads both sides, understands the intent, and produces a coherent result. It's like a senior engineer who sits down with two PRs and figures out how they fit together.

Human review. Sometimes the conflict is semantic, not syntactic. The code merges cleanly but the *logic* contradicts itself. Two agents made incompatible design decisions. No automated merge will catch this. This is where you earn your keep as the engineer. Review the branches before merging. Read the diffs side by side. Make sure the pieces actually fit.

In practice, you'll use all three. Prevent what you can. Automate the rest. Review everything.

12.5 Orchestration Overhead

More agents is not always better.

Every additional agent adds coordination cost. You have to decompose the work. Set up worktrees. Define interfaces. Handle merges. Review multiple branches instead of one. For a task that takes a single agent thirty minutes, spinning up three agents might take forty-five — twenty minutes of parallel agent work plus twenty-five minutes of your time orchestrating.

The break-even point is higher than you think. In my experience, multi-agent orchestration starts paying off when the total task would take a single agent *at least two hours*. Below that, the overhead eats the gains.

There are other costs too. Context gets fragmented. Each agent sees only its piece. No single agent understands the whole system the way one agent working alone would. This can lead to inconsistencies — different naming conventions, different error handling patterns, different assumptions about shared state.

The skill isn't running as many agents as possible. The skill is knowing *when* to parallelise and when a single focused agent is the right tool. A crew of five isn't always faster than one experienced sailor who knows the boat.

12.6 A Practical Example

Let's make this concrete. You're building a web app and you need to add a new feature: user notifications. Users should see a bell icon with a count, click it to see a list, and mark notifications as read. You need an API, a frontend component, and tests.

This is a textbook case for three parallel agents.

Step 1: Define the interface. Before spinning up any agents, you spend five minutes writing down the API contract. `GET /api/notifications`

returns a list. `PATCH /api/notifications/:id` marks one as read. The notification object has `id`, `message`, `read`, `created_at`. Write this in a shared document or a stub file. This is the contract all three agents work against.

Step 2: Create the worktrees.

```
git worktree add ../app-api agent/notifications-api
git worktree add ../app-frontend agent/notifications-frontend
git worktree add ../app-tests agent/notifications-tests
```

Step 3: Launch the agents. Each agent gets a clear, scoped task:

- Agent 1: "Implement the notifications API endpoints in `../app-api`. Follow the contract in `API_CONTRACT.md`. Include model, route, and controller. Write unit tests for the controller."
- Agent 2: "Build the notifications UI component in `../app-frontend`. It calls `GET /api/notifications` and `PATCH /api/notifications/:id`. Show a bell icon with unread count. Clicking opens a dropdown list."
- Agent 3: "Write integration tests in `../app-tests`. Cover the full flow: create a notification, fetch the list, mark as read, verify the count updates."

Step 4: Let them work. The three agents run simultaneously. Each commits to its own branch. You monitor progress but don't intervene unless something goes wrong.

Step 5: Review and merge. When all three finish, you review each branch. The API branch gets merged first — it's the foundation. Then the frontend branch. Then the tests. You run the full test suite after each merge to catch integration issues early.

Total time: maybe forty minutes. The API agent took thirty, the frontend agent took twenty-five, and the test agent took thirty-five. But they ran in parallel, so wall-clock time was thirty-five minutes plus ten minutes of your orchestration work.

A single agent doing everything sequentially? Probably ninety minutes.

The maths works when the task is big enough. And as you get better at decomposition, you'll develop an intuition for which tasks are worth

splitting and which aren't. Like most things in engineering, it's a judgement call. But now you have the tools to make it.

13 When Agents Get It Wrong

Every engineer who works with agents long enough has a collection of these stories. The moments where you lean back in your chair, stare at the screen, and mutter something unprintable. They're humbling. They're educational. And they're *inevitable*.

This chapter is a collection of war stories — things that actually go wrong when you hand real work to an agent. Not hypotheticals. Not contrived demos. The kind of failures that cost you an afternoon, a deploy, or your faith in automation. Each one maps back to principles from earlier chapters, because principles are cheap until you learn them the hard way.

Read these before you make the same mistakes. Or read them after, and feel less alone.

13.1 The Eager Refactorer

The task was simple: fix a z-index issue on a dropdown menu. The dropdown was rendering behind a modal overlay. A one-line CSS fix — maybe two if you're being careful.

The agent saw the dropdown component, decided its CSS was "inconsistent with modern practices," and refactored it. Then it noticed the modal used a different styling pattern, so it refactored that too. Then the button components. Then the card components. Before you looked up from your coffee, thirty-two files had changed. The diff was 1,400 lines. The agent had essentially rewritten the component library's styling system, migrating from one approach to another with admirable consistency and zero authorization.

The original z-index bug? Still there. Buried somewhere in the avalanche of changes, the agent had reproduced the same layering issue with new class names.

The PR was unreviewable. You can't meaningfully review 1,400 lines of CSS changes across thirty-two files. You *can* delete the branch and start over. Which is what happened.

What you do differently now: Scope tasks tightly. "Fix the z-index on the dropdown in `NavMenu.tsx`, touch nothing else." Use a dedicated branch so the damage is contained. And *always* check the diff before you let the agent move on to anything else. The moment you see the file count climbing past what makes sense for the task, stop the agent. The guardrails chapter exists for a reason.

13.2 The Hallucinated Library

The agent needed to parse some complex date ranges from user input. It imported `@temporal/daterange-parser`, wrote elegant code using its `.parseRange()` method with options for locale-aware parsing and fuzzy matching. The code was clean. The types were correct. The error handling was thoughtful. It even wrote tests — which, naturally, also imported the hallucinated package.

Everything looked perfect in the diff. The function signatures made sense. The API was well-designed. It was *such* a plausible library that you almost didn't question it.

Then `npm install` failed. The package didn't exist. Had never existed. The agent had invented a library, given it a believable name and a coherent API, and written production code against a fiction.

The insidious part: if you'd only done code review — reading the logic, checking the types, evaluating the approach — you would have approved it. The code was *good*. It just didn't connect to reality.

What you do differently now: The agent runs the tests. Always. If your setup doesn't allow that, you run them yourself before reviewing code. No exceptions. A failing `npm install` is a loud, obvious signal. A hallucinated API that was never executed is a silent bomb. Tests catch what human review misses — not because your review is bad, but because plausible fictions are *designed* to pass review. That's what hallucination *is*.

13.3 The Infinite Loop

You asked the agent to fix a failing integration test. It read the error, made a change, ran the test. New error. It read *that* error, made another change, ran the test. Different error. Then a variation of the first error. Then something new. Then the first error again.

You were in another terminal, working on something else. Forty minutes later you checked back. The agent had made nineteen attempts. It had burned through 200,000 tokens. The code was now worse than when it started — a geological record of failed fixes layered on top of each other. The agent was still going, cheerfully confident that attempt twenty would be the one.

It wasn't.

The fundamental problem was that the agent didn't understand *why* the test was failing. It was pattern-matching on error messages and making local edits, but the actual issue was an architectural misunderstanding — a shared database state between test cases that required a different setup approach entirely. No amount of tweaking the code under test would fix a problem in the test harness.

What you do differently now: Set iteration limits. Three attempts at the same problem is a reasonable maximum. If the agent hasn't solved it in three passes, it won't solve it in thirty. When you see the loop — error, fix, different error, fix, original error — *break it manually.* Stop the agent, read the errors yourself, and either give it a completely different approach or take over. The agent's time is cheap. Your wasted afternoon is not. More importantly, start a fresh context. The agent's understanding is now polluted with nineteen wrong theories. A clean start with your diagnosis of the *actual* problem will get further, faster.

13.4 The Confident Wrong Answer

A background job was occasionally processing the same item twice. Classic race condition. You pointed the agent at the problem.

The agent analysed the code, identified the race window, and added a 500ms delay between the check and the update. "This ensures the previous transaction has time to commit before the next check occurs," it explained, with the serene confidence of someone who has never operated a system under load.

Tests passed. The delay was longer than the test's transaction time, so the race window closed — in the test environment, with one concurrent worker, on a quiet machine.

It went to production. Under load, with twelve workers and variable database latency, 500ms was sometimes not enough. And now you had a *new* problem: the delay had reduced throughput enough that the job queue backed up during peak hours, creating cascading timeouts that took down an unrelated service.

The sleep didn't fix the race condition. It hid it at low concurrency and made the system worse at high concurrency. A proper fix — an advisory lock or an idempotency key — would have been correct at any scale. The agent chose the fix that made the test green, not the fix that was *right*.

What you do differently now: You treat passing tests as necessary but not sufficient. When an agent fixes a concurrency issue, you ask *yourself* whether the fix is correct under load, under failure, under conditions the test suite doesn't cover. Agents optimise for the feedback signal you give them, and if that signal is "tests pass," they'll find the shortest path to green — even if that path is a `time.Sleep`. Your engineering judgment about *why* something works matters more than *whether* the tests pass. This is the part of the job that hasn't been automated yet. Lean into it.

13.5 The Context Amnesia

Two hours into a session, you had a nicely evolving feature. You'd told the agent early on: "Don't use any ORM — we're writing raw SQL in this project. That's a deliberate choice." The agent acknowledged this and wrote clean, handcrafted queries for the first several tasks.

Then you asked it to add a new endpoint. Ninety minutes of context had accumulated. The agent built the endpoint — using Prisma. Full schema

file, migration, generated client. Beautiful code. Completely contradicting the constraint you'd established at the start of the session and the patterns in every other file it had written that day.

When you pointed this out, the agent apologised, rewrote everything in raw SQL, and acted as if nothing had happened. It hadn't *decided* to ignore your constraint. It had simply lost it. The context window had filled with enough intermediate work that the early instruction had faded into irrelevance.

What you do differently now: Long sessions degrade. This is a fundamental property of how context windows work, not a bug that will be patched next quarter. Keep tasks short and focused. Commit working code at natural boundaries so progress is captured in git, not just in the conversation. Start fresh sessions for fresh tasks. And for project-wide constraints like "no ORM" or "no new dependencies," put them in a `CLAUDE.md` file or equivalent that the agent reads at startup. Don't rely on the agent *remembering* what you said two hours ago. It won't. Write it down.

13.6 The Dependency Avalanche

You asked for a date picker. A simple input where users can select a date. The agent evaluated the options and decided to be thorough.

It installed `moment.js` for date handling. Then `@popperjs/core` for positioning the dropdown. Then a full UI component library because "it provides accessible date picker primitives." Then a CSS preprocessor because the component library's theme system required it. Then two utility packages that the component library's date picker needed as peer dependencies.

Six new dependencies. Your bundle size went from 180KB to 540KB. Your build time doubled. You had a date picker, though. It was very nice.

The native HTML `<input type="date">` would have been fine. Or a single lightweight picker library at 8KB. Instead you'd inherited an entire ecosystem because the agent optimised for completeness rather than minimality.

The worst part wasn't the bundle size — it was the maintenance surface. Six new packages means six new things that can have security vulnerabilities, six new things that can break on upgrade, six new changelogs to read when Dependabot starts filing PRs. You didn't just add a date picker. You adopted six open-source projects.

What you do differently now: Constrain what agents can install. "No new dependencies without asking me first" is a legitimate and often *wise* instruction. When you do allow new packages, tell the agent your constraints: bundle budget, no packages with fewer than N weekly downloads, no packages that pull in transitive mega-dependencies. Agents don't have opinions about bundle size or maintenance burden. They don't maintain the project next year. *You* do. So you set the boundaries.

13.7 The Common Thread

Every one of these stories has the same root cause: the agent was doing *exactly what it was designed to do*, and the human wasn't providing enough structure.

The eager refactorer was being helpful. The hallucinated library was being creative. The infinite loop was being persistent. The confident wrong answer was being test-driven. The context amnesia was a limitation, not a choice. The dependency avalanche was being thorough.

None of these are agent bugs. They're *workflow* bugs. The fix is never "use a smarter agent." The fix is always the same: tighter scope, better feedback loops, more structure, shorter sessions, and a human who stays engaged.

Agents get things wrong. So do humans. The difference is that humans get things wrong slowly enough to notice. Agents get things wrong at the speed of autocomplete, and by the time you look up from your coffee, thirty-two files have changed.

Stay in the loop. Check the diffs. Trust but verify. And when things go sideways — because they will — remember that the delete-branch-and-start-over button is the most underrated tool in your workflow.

14 When Not to Use Agents

This book is about using agents well. This chapter is about knowing when not to use them at all.

If you've read this far, you're probably sold on agentic engineering. Good. But the fastest way to lose credibility — and waste time — is to reach for an agent when the job calls for a human. The best agentic engineers aren't the ones who use agents the most. They're the ones who know *exactly* when to put the agent away and do the work themselves.

14.1 The Overhead Tax

Every agent interaction has a cost. You write the prompt. You wait for the output. You review what it produced. You fix the mistakes. You re-run if it missed the point. That's overhead, and it's real.

For complex tasks that would take you an hour, spending two minutes on prompt and review is a bargain. But for tasks you can do in thirty seconds? The overhead makes agents *slower*, not faster.

Renaming a variable. Fixing a typo. Tweaking a config value. Adding a log line. These are muscle-memory tasks. Your fingers know the keystrokes. By the time you've typed a prompt describing what you want, you could have already done it.

This isn't a failure of agents. It's arithmetic. Small tasks have small payoffs, and the fixed cost of agent interaction eats the margin. Don't let the novelty of agents trick you into using them for everything. Some work is just faster by hand.

14.2 Novel Architecture Decisions

When you're designing a system from scratch — choosing between a monolith and microservices, deciding on your data model, picking your communication patterns — the *thinking is the work*. The value isn't in the diagram or the document. The value is in the mental model you build while wrestling with the tradeoffs.

An agent can help you explore options. It can list the pros and cons of event sourcing versus CRUD. It can sketch out what a particular architecture might look like. That's useful as input.

But delegating the architecture itself to an agent means you don't understand your own system. You'll struggle to debug it, extend it, or explain it to your team. The senior engineer's job is to make the hard calls — to weigh the tradeoffs that don't have clean answers, to decide what complexity is worth carrying and what isn't. That judgment comes from doing the work, not from reading an agent's summary of it.

Use agents as a sparring partner for architecture. Not as the architect.

14.3 Security-Critical Code

Authentication flows. Encryption. Access control. Input validation. Token handling. These are areas where "looks correct" isn't good enough.

Security bugs are different from regular bugs. A broken sort function produces wrong output that someone notices. A broken auth check produces *no visible symptoms* until an attacker finds it. The code looks fine. The tests pass. And six months later you're writing an incident report.

Agents produce plausible code. That's their strength and, in security contexts, their danger. A subtle flaw in a JWT validation flow, a missing check on a redirect URL, a timing side-channel in a password comparison — these are the kinds of mistakes that survive code review because they *look right*.

Write security-critical code yourself. Review it carefully. Get a second pair of human eyes on it. If you do use an agent to draft security code, treat

that draft with more suspicion than you'd give a junior developer's first attempt, not less.

14.4 When You Need to Learn

You're picking up a new language. A new framework. A new paradigm. The temptation is obvious: let the agent write the code while you focus on the big picture.

Resist it.

If you let the agent write all the Rust while you're learning Rust, you haven't learned Rust. You've built something you can't maintain, debug, or extend without the agent. You've created a dependency, not a skill.

There's a crucial difference between using an agent to *explain* something and using it to *do* something. Asking "why does this borrow checker error occur?" builds understanding. Asking "fix this borrow checker error" doesn't.

When the goal is learning, slow down. Write the code yourself. Make the mistakes yourself. Use agents as tutors, not ghostwriters. The understanding you build by struggling through the hard parts is the whole point.

14.5 Emotionally Charged Decisions

Not every engineering decision is technical. Some of the hardest ones are human.

Deprecating an API that a partner depends on. Telling a stakeholder their feature request won't make the cut. Pushing back on a deadline you know is unrealistic. Deciding to sunset a product that still has users.

These decisions require empathy. They require reading the room, understanding the politics, weighing the human cost alongside the technical cost. They require *accountability* — someone who will own the decision and its consequences.

An agent can draft the email. It can help you think through the talking points. But the decision itself, and the conversation that delivers it, must come from a human. People deserve to hear hard news from a person, not from someone who copy-pasted an agent's output.

14.6 When the Codebase Is Too Messy

Agents amplify what's already there. In a clean, well-structured codebase with strong conventions, agents produce clean, well-structured code. They pick up the patterns and follow them.

In a mess? They produce more mess.

If your codebase has inconsistent naming, tangled dependencies, no clear module boundaries, and three different ways to do the same thing — an agent will pick up *all* of those patterns. It might combine the worst parts of each. It doesn't know which patterns are intentional and which are technical debt. It just sees what's there and produces more of it.

Sometimes the right move is to clean up before you bring agents in. Refactor the module. Establish the convention. Delete the dead code. Make the codebase a place where an agent can do good work. This is unglamorous, manual labor. But it's the foundation that makes everything else possible.

Think of it like a workshop. You don't hand power tools to someone in a cluttered, disorganized shop. You clean up first, *then* bring in the tools.

14.7 The Craft Argument

There's one more reason to sometimes put the agent away, and it's not about efficiency or risk. It's about craft.

Writing code by hand builds something that agents can't give you. Muscle memory. Pattern recognition that lives in your fingers, not just your head. The deep, intuitive understanding that comes from having written the same kind of function dozens of times. The quiet satisfaction of solving a hard problem through your own reasoning.

These things matter. Not because they're romantic, but because they make you a better engineer. The developer who has hand-written a parser understands parsing differently than one who has only prompted an agent to write one. The developer who has debugged a concurrency issue at 2am *feels* the danger of shared mutable state in a way that reading about it never provides.

Agents are tools. Powerful ones. But if you let them do all the work, your own skills atrophy. And when you hit a problem that the agent can't solve — and you will — you need those skills to still be sharp.

Stay in practice. Write code by hand regularly. Not because it's faster, but because it keeps you dangerous.

14.8 The Judgment That Matters

The central skill of agentic engineering isn't prompting. It isn't tool configuration. It isn't knowing which model to use for which task.

It's *judgment*.

Knowing when an agent will save you an hour and when it will waste one. Knowing when to trust the output and when to rewrite it from scratch. Knowing when to delegate and when to dig in yourself.

The best agentic engineers use agents aggressively — but selectively. They reach for agents when the leverage is real: large-scale refactors, boilerplate generation, code exploration, test writing, documentation. And they put the agent away when the work requires human thinking, human accountability, or human craft.

That judgment is what separates agentic engineers from prompt jockeys. Anyone can type a prompt. Knowing *when not to* is the harder skill.

15 Agentic Teams

Software has always been a team sport. Agents don't change that. But they change the shape of the team, the rhythm of the work, and the skills that matter. If you're leading a team — or working on one — you need to think about this now, not later.

15.1 The 10x Multiplier Is Real, but Distributed Differently

You've heard the claim: agents make engineers 10x more productive. It's not wrong. It's just misleading.

Agents make certain tasks *dramatically* faster. Generating boilerplate, writing test scaffolding, refactoring across a hundred files, migrating API versions, wiring up CRUD endpoints — these go from hours to minutes. The speed gain is real and it's massive.

But other tasks barely change. System design still requires thinking. Debugging a subtle race condition still requires patience, intuition, and deep understanding of the runtime. Stakeholder conversations still take however long they take. The agent can't sit in your architecture review and ask the right questions.

The 10x multiplier isn't a flat multiplier across your day. It's a spike graph. Some tasks go 50x faster. Some go 1x. A few might even slow down if you fight the agent instead of doing it yourself.

Teams that understand this deploy agents strategically. They don't hand every task to an agent and expect magic. They identify the high-leverage zones — the tasks where agents genuinely compress timelines — and they focus agent effort there. The rest stays human.

15.2 Code Review Changes

Here's what happens when agents enter a team's workflow: PR volume goes up. *Way* up. An engineer who used to open two PRs a day now opens five. The code in those PRs is syntactically correct, well-formatted, and passes tests. And reviewing it is *exhausting*.

The old model of code review — read every line, check for off-by-one errors, verify edge case handling — doesn't scale when half the code was generated in seconds. You'll burn out your reviewers in a week.

The shift is from "is this correct?" to "is this the right approach?" Agent-generated code is usually *locally* correct. It does what was asked. The question is whether what was asked was the right thing. Does this new service need to exist, or should this logic live in the existing one? Is this the right abstraction boundary? Does this change make the system simpler or more complex?

Reviewers become architects. They zoom out. They check intent, not implementation.

Practical adaptations that work:

- Smaller, more focused PRs — easier for both agents and reviewers
- Automated checks handle the mechanical stuff (linting, test coverage, type checking)
- Review time is protected on the calendar, not squeezed between meetings
- Teams agree on "trusted patterns" — if a PR follows a known pattern and passes CI, it gets a faster review track

Review fatigue is the silent killer of agent-assisted teams. Take it seriously.

15.3 The Junior Engineer Question

This is the hardest problem in this chapter, and I don't have a clean answer.

Junior engineers have traditionally learned by doing the work that agents now do faster and better. Writing that first CRUD endpoint. Wrestling with a tricky CSS layout. Figuring out why the test is failing. These tasks

were tedious for seniors but *formative* for juniors. The struggle was the education.

If agents handle all of that, where does the learning happen?

The worst outcome is juniors who become prompt-dependent — they can get an agent to generate code, but they can't explain what the code does or debug it when it breaks. They skip the understanding phase entirely. That's not engineering; that's a very expensive copy-paste workflow.

The better path is using agents as *tutors*, not replacements. A junior writing a database migration should ask the agent to *explain* the migration, not just generate it. "Why did you use a transaction here?" "What happens if this fails halfway?" "Show me what this looks like without the ORM." The agent becomes a patient teacher with infinite time — something most seniors can't offer.

Pair programming with agents, *supervised by seniors*, is the most promising model I've seen. The junior drives the agent. The senior watches, asks questions, and intervenes when the junior accepts something they don't understand. It's slower than letting the agent do everything, but it produces engineers who actually know what they're doing.

Teams that skip this investment are borrowing from the future. Today's unsupervised juniors are tomorrow's seniors who can't debug production without an AI crutch.

15.4 Knowledge Distribution

Here's a pattern that shows up in every team that adopts agents unevenly: one engineer gets fluent with agents, starts shipping at 3x the rate of everyone else, and within a few months has touched every part of the codebase. They become the single point of failure.

The bus factor drops to one. Not because anyone planned it, but because velocity and knowledge concentration are correlated. The engineer who ships the most learns the most. The others fall behind, not just in output but in *understanding* of the system they collectively own.

This is a management problem, not a technology problem. The fix is structural:

- **Shared `CLAUDE.md` files.** Every project has one. Everyone contributes to it. It encodes the team's collective knowledge, not one person's.
- **Shared workflows and conventions.** The team agrees on how they use agents — which tools, which patterns, which guardrails. No lone-wolf setups.
- **Rotation.** Agent-fluent engineers rotate to different parts of the codebase. Knowledge spreads through work, not documentation.
- **Agent session sharing.** Some teams have started sharing interesting agent sessions — the prompts, the outputs, the decisions. It's a form of knowledge transfer that didn't exist before.

The goal isn't to slow down your fastest engineer. It's to make sure the team's knowledge keeps up with the team's output.

15.5 Shared Conventions Matter More

We covered conventions in an earlier chapter. In a team context, the stakes are higher.

When a solo engineer uses agents, their conventions affect one person. When a team uses agents, conventions affect *every agent session across the entire team.* A well-structured project with clear naming, consistent patterns, and a maintained `CLAUDE.md` means every engineer's agents start from a strong foundation.

A messy project means every agent reinvents the wheel. Different engineers get different outputs. The codebase drifts. Reviews get harder because you're now reviewing not just the code but the *style* of the code, which varies by which engineer's agent wrote it.

Coding standards in an agentic team aren't about aesthetics. They're about *agent effectiveness.* The team that agrees on project structure, naming conventions, testing patterns, and documentation format gets better output from every agent session. It's a force multiplier on a force multiplier.

Invest the time. Write the standards down. Enforce them in CI. It pays back on every single PR.

15.6 The New Standup

What does daily coordination look like when each engineer is running multiple agent sessions in parallel?

The old standup: "Yesterday I worked on the auth refactor. Today I'll continue. No blockers."

The new standup: "I have three agents running. The auth refactor landed and is in review. The API migration is 80% done — the agent got stuck on the legacy XML parsing, so I'm taking that over manually. I just kicked off a third session to generate integration tests for the billing module."

The granularity changes. Work moves faster, so coordination needs to keep up. An engineer might *start and finish* a task between standups. The daily sync becomes less about progress updates and more about intent alignment — making sure three engineers aren't all sending agents after the same problem from different angles.

Some teams are moving to async standups with more frequent check-ins. Others use shared dashboards that track active agent sessions. The right answer depends on the team, but the old rhythm of "one update per person per day" often isn't enough.

15.7 Hiring Changes

If agents handle the mechanical parts of coding, what do you actually need from an engineer?

Raw coding speed matters less. The engineer who could bang out a perfect binary search on a whiteboard in three minutes has a skill that agents have commoditized. That's not worthless — understanding algorithms still matters — but it's no longer the differentiator.

What matters more:

- **System design.** The ability to decompose a problem into components, define interfaces, and reason about tradeoffs. Agents can implement a design. They can't tell you which design is right for your constraints.
- **Judgment.** Knowing when to use the agent and when to think. Knowing when the agent's output is wrong even though it looks right. Knowing which corners to cut and which to protect.
- **Communication.** The ability to articulate intent clearly — to agents, to teammates, to stakeholders. Vague thinkers get vague agent output. Precise thinkers get precise results.
- **Problem decomposition.** Breaking a large task into agent-sized pieces is a skill. It's related to system design but more tactical. The engineer who can turn a Jira epic into ten well-scoped agent prompts will outperform the one who pastes the whole epic into a chat window.

The interview that asks "implement this algorithm on a whiteboard" is testing a skill that matters less every year. The interview that asks "here's a system with these constraints — walk me through how you'd build it, what tradeoffs you'd make, and how you'd verify it works" is testing the skills that matter *more* every year.

Hire for judgment. You can always give them an agent for the rest.

16 Final Words

This book started as notes to myself. Things I was learning, mistakes I was making, patterns I was noticing. I wrote them down because I needed to make sense of what was happening — to my craft, to my workflow, to the way I think about building software.

Somewhere along the way, the notes became chapters, and the chapters became this book.

16.1 What I Believe

I believe the craft of software engineering is not dying. It's shedding its skin. The part that was always mechanical — the typing, the boilerplate, the repetitive plumbing — that's being absorbed by machines. What remains is the part that was always human: the judgement, the taste, the ability to look at a system and know what it *should* be.

I believe agents are not replacements for engineers. They are amplifiers. A mediocre engineer with agents produces mediocre software faster. A great engineer with agents produces great software at a pace that was previously impossible. The differentiator was never typing speed. It was always thinking.

I believe the engineers who thrive in this new world will be the ones who invest in the fundamentals — context management, guardrails, testing, conventions, clear communication — rather than chasing the latest tool. Tools change every six months. Principles endure.

16.2 What I Got Wrong

I'm sure parts of this book are already wrong. The landscape moves fast. A technique I swear by today might be obsolete by the time you read this. A tool I recommended might have been replaced by something better.

That's fine. The specific tools were never the point. The point is the *thinking* — how to reason about autonomy, how to structure work for agents, how to maintain your own judgement while leveraging theirs. If I've helped you develop that thinking, the book did its job, even if every code example is outdated.

16.3 The Crew Metaphor, One Last Time

You're the captain. The agents are your crew. The ship is whatever you're building — a product, a system, a company, a career.

A good captain doesn't do everything alone. That's not strength — it's a bottleneck. A good captain also doesn't hand over the wheel and go below deck. That's not delegation — it's abdication.

The art is in the middle. You set the course. You make the calls that matter. You trust your crew with the work they're good at, and you stay close enough to intervene when the sea gets rough. You build trust over time, task by task, commit by commit.

Some days the crew will surprise you with how good they are. Other days they'll do something baffling and you'll wonder why you bothered. Both are normal. The relationship gets better with practice, like any working relationship does.

16.4 Go Build Something

The best way to learn agentic engineering is to do it. Not to read about it — though I hope this book gave you a useful foundation — but to open a terminal, point an agent at a real problem, and watch what happens.

Start small. Fix a bug. Write some tests. Refactor a module. Pay attention to what works and what doesn't. Adjust your workflow. Try again.

Then go bigger. Run two agents in parallel. Set up worktree isolation. Build a pipeline. Break something, learn from it, build a guardrail.

The engineers who will define this era aren't waiting for permission or certainty. They're experimenting, shipping, and learning — with a crew at their side.

I hope you'll be one of them.

Rasmus Bornhøft Schlünsen
March 2026